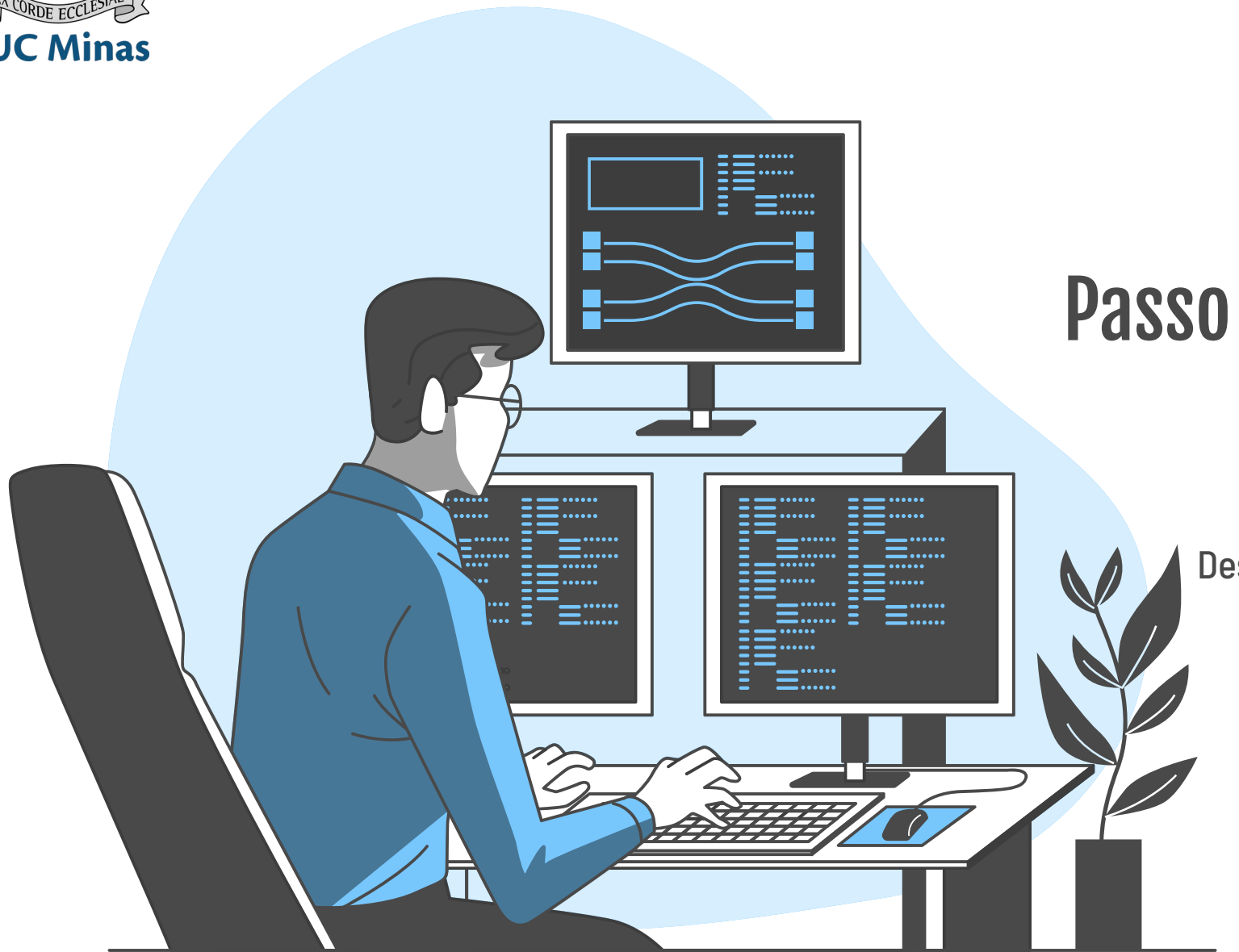
PUC Minas



Aula 05 · FullStack

Passo a passo: CRUD full stack com Supabase

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1. Construir o back-end completo: entidade, repository, service, controller, CORS
2. Testar a API antes de tocar no front
3. Construir o front: service, formulário, lista e edição
4. Exibir os erros de validação vindos do Spring
5. Conferir os dados no Table Editor do Supabase

Antes de começar

- Back-end, aula 06 (Supabase)
- Aulas 01 a 04 desta pasta

Duração sugerida

2 aulas de 100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 O que vamos construir

02 Parte A: back-end

03 Parte B: front-end

04 Aprofundando: deixando o CRUD profissional

01

O que vamos construir

O que vamos construir

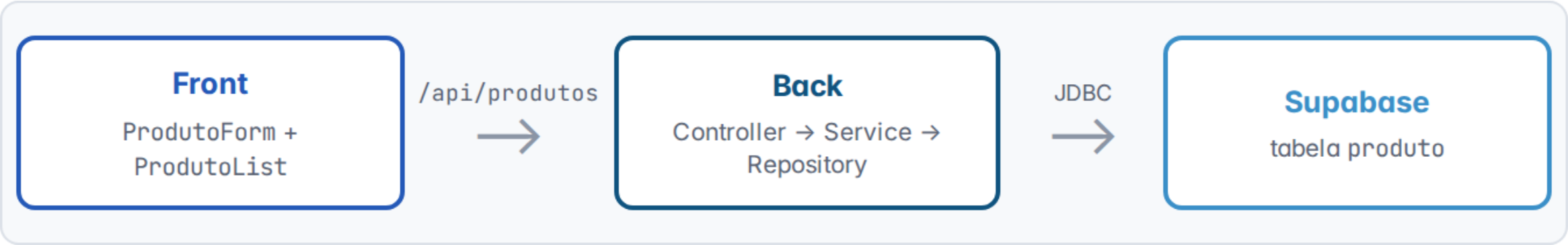
Uma versão simplificada do projeto `AraCoffee`: o cardápio de uma cafeteria. O `AraCoffee` original usa MongoDB Atlas; aqui faremos com **Supabase**, para você ter um exemplo full stack com banco relacional. Trocar para MongoDB muda só o `model` e o `repository` (Back-end, aula 07).

01

O que vamos construir

Aula 05 · FullStack

O que vamos construir



Arquitetura do cardápio.

02

Parte A: back-end

Passo 1: projeto e dependências

No Spring Initializr: Artifact **Back**, Java **25**, dependências **Spring Web**, **Spring Data JPA**, **PostgreSQL Driver**, **Validation**, **DevTools**. Configure o `application.properties`:

Back/src/main/resources/application.properties

```
spring.application.name=Back
server.port=${PORT:8080}

spring.datasource.url=${DB_URL:jdbc:postgresql://aws-0-sa-east-1.pooler.supabase.com:5432/postgres}
spring.datasource.username=${DB_USER:postgres.abcdefghijklmnop}
spring.datasource.password=${DB_PASSWORD}
spring.datasource.hikari.maximum-pool-size=5
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false

app.cors.allowed-origins=${CORS_ORIGINS:http://localhost:5173}
```

Passo 2: entidade e repository

model/Produto.java

```java
@Entity
@Table(name = "produto")
public class Produto {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  @Column(nullable = false, length = 80)
  private String nome;

  @Column(nullable = false)
  private BigDecimal preco;

  @Enumerated(EnumType.STRING)
  private Categoria categoria;   // CAFE, CHA, DOCE

  private boolean disponivel = true;

  protected Produto() { }
  // getters e setters
}
```

model/Categoria.java

```java
public enum Categoria { CAFE, CHA, DOCE }
```

repository/ProdutoRepository.java

```java
public interface ProdutoRepository
    extends JpaRepository<Produto, Long> {

  List<Produto> findByCategoriaOrderByNomeAsc(
      Categoria categoria);

  List<Produto> findAllByOrderByNomeAsc();
}
```

Passo 3: DTO com validação e service

dto/ProdutoDTO.java

```java
public record ProdutoDTO(
    @NotBlank(message = "O nome é obrigatório.") @Size(max = 80) String nome,
    @NotNull(message = "O preço é obrigatório.") @Positive(message = "O preço deve ser maior que zero.") BigDecimal preco,
    @NotNull(message = "Escolha uma categoria.") Categoria categoria,
    boolean disponivel) { }
```

Passo 3: DTO com validação e service

service/ProdutoService.java

```java
@Service
public class ProdutoService {
    private final ProdutoRepository repo;
    public ProdutoService(ProdutoRepository repo) { this.repo = repo; }

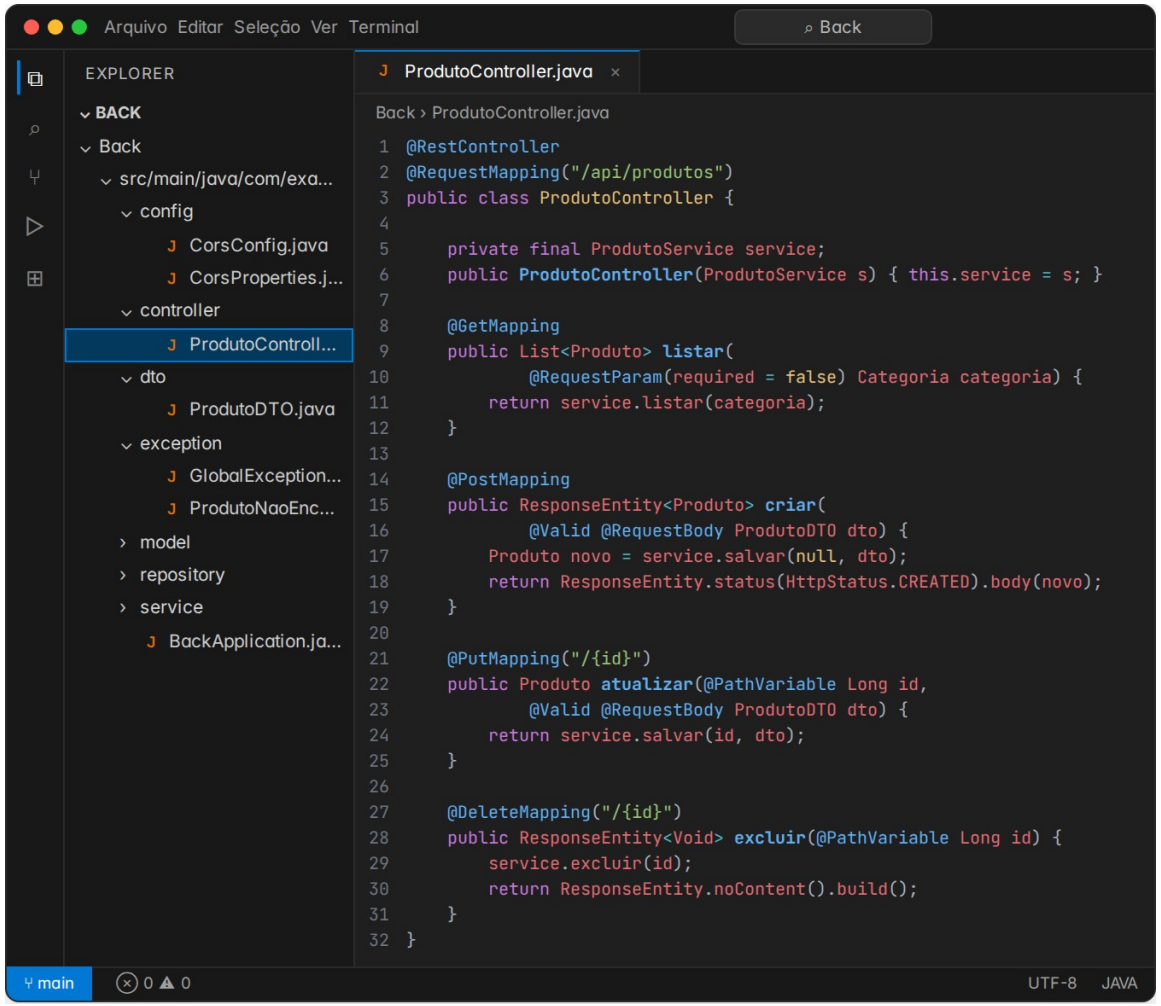
    public List<Produto> listar(Categoria c) {
        return c == null ? repo.findAllByOrderByNomeAsc() : repo.findByCategoriaOrderByNomeAsc(c);
    }

    public Produto buscar(Long id) {
        return repo.findById(id).orElseThrow(() -> new ProdutoNaoEncontradoException(id));
    }

    @Transactional
    public Produto salvar(Long id, ProdutoDTO dto) {
        Produto p = (id == null) ? new Produto() : buscar(id);
        p.setNome(dto.nome().trim());
        p.setPreco(dto.preco());
        p.setCategoria(dto.categoria());
        p.setDisponivel(dto.disponivel());
        return repo.save(p);
    }

    public void excluir(Long id) { repo.delete(buscar(id)); }
}
```

Passo 4: controller, erros e CORS



Controller do cardápio. Copie `CorsConfig`, `CorsProperties` e o `GlobalExceptionHandler` da aula 03 e do Back-end, aula 05.

Passo 4: controller, erros e CORS

O `@Valid` lança `MethodArgumentNotValidException` quando o DTO é inválido. Acrescente um tratador para devolver a **primeira mensagem** no mesmo formato de erro:

exception/GlobalExceptionHandler.java (trecho)

```java
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ErroResposta> validacao(MethodArgumentNotValidException ex, HttpServletRequest req) {
    String msg = ex.getBindingResult().getFieldErrors().stream()
            .map(FieldError::getDefaultMessage).findFirst().orElse("Dados inválidos.");
    return montar(HttpStatus.BAD_REQUEST, msg, req);
}
```

Passo 5: testar a API sozinha

Terminal

```
aluno@pc:~/cardapio/Back$ DB_PASSWORD='***' ./mvnw spring-boot:run
HikariPool-1 - Start completed.
Started BackApplication in 4.1 seconds

aluno@pc:~/cardapio/Back$ curl -s -X POST localhost:8080/api/produtos -H 'Content-Type: application/json' \
    -d '{"nome":"Espresso","preco":7.5,"categoria":"CAFE","disponivel":true}'
{"id":1,"nome":"Espresso","preco":7.5,"categoria":"CAFE","disponivel":true}
aluno@pc:~/cardapio/Back$ curl -s -X POST localhost:8080/api/produtos -H 'Content-Type: application/json' -d
'{"nome":""}'
{"status":400,"erro":"Bad Request","mensagem":"O nome é obrigatório.","caminho":"/api/produtos",...}
```

Sempre teste o back sozinho antes de integrar. Se aqui não funciona, no React também não vai.

03

Parte B: front-end

Passo 6: projeto e camada de comunicação

Na raiz: `npm create vite@latest Front -- --template react`. Copie `config/apiConfig.js` e `services/httpClient.js` da aula 04 e crie o service:

Front/src/services/produtoService.js

```
import { requisicao } from "./httpClient";
const C = "/api/produtos";

export const listarProdutos = (categoria = "") => requisicao(categoria ? `${C}?categoria=${categoria}` : C);
export const criarProduto = (p) => requisicao(C, { method: "POST", body: JSON.stringify(p) });
export const atualizarProduto = (id, p) => requisicao(`${C}/${id}`, { method: "PUT", body: JSON.stringify(p) });
export const excluirProduto = (id) => requisicao(`${C}/${id}`, { method: "DELETE" });
```

Passo 7: formulário que também edita

Front/src/components/ProdutoForm.jsx

```jsx
import { useEffect, useState } from "react";

const VAZIO = { nome: "", preco: "", categoria: "CAFE", disponivel: true };

export default function ProdutoForm({ emEdicao, onSalvar, onCancelar }) {
  const [form, setForm] = useState(VAZIO);

  // quando o pai escolhe um produto para editar, preenche o formulário
  useEffect(() => { setForm(emEdicao ?? VAZIO); }, [emEdicao]);

  const mudar = (e) => {
    const { name, value, type, checked } = e.target;
    setForm({ ...form, [name]: type === "checkbox" ? checked : value });
  };

  const enviar = async (e) => {
    e.preventDefault();
    const ok = await onSalvar({ ...form, preco: Number(form.preco) });
```

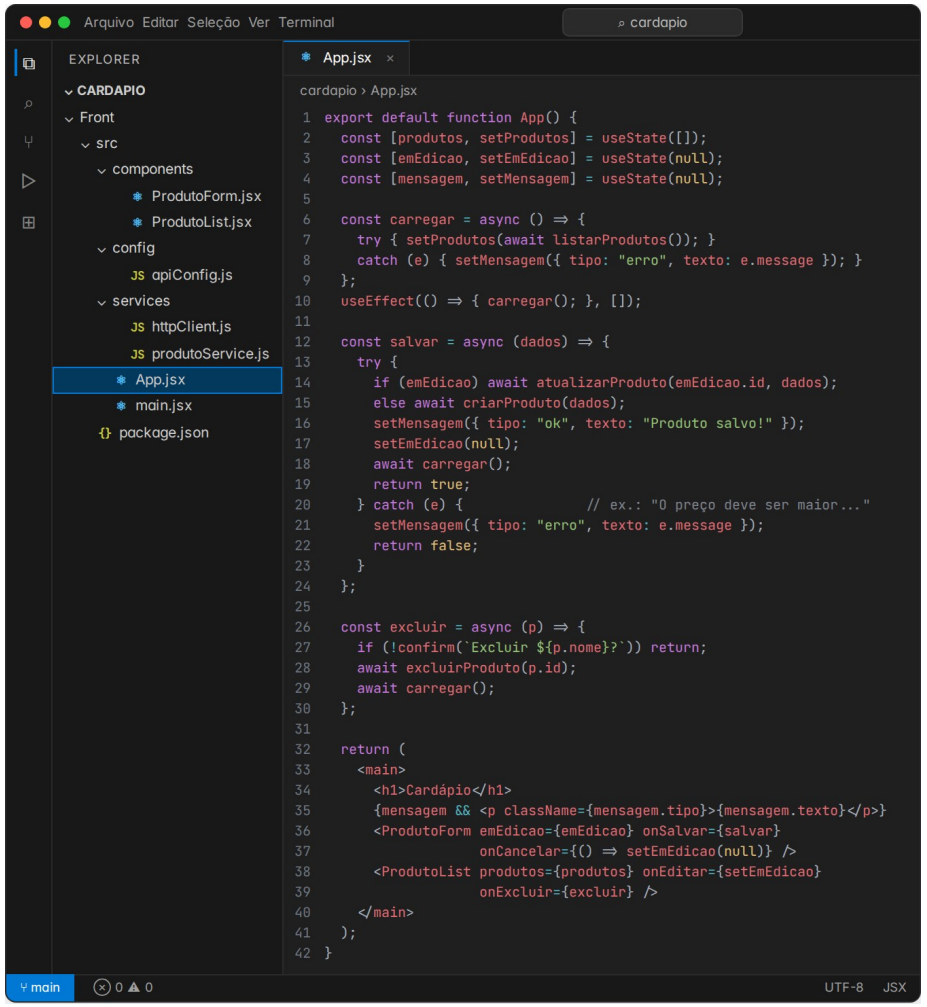
Passo 7: formulário que também edita (cont.)

Front/src/components/ProdutoForm.jsx

```
    if (ok) setForm(VAZIO);
  };

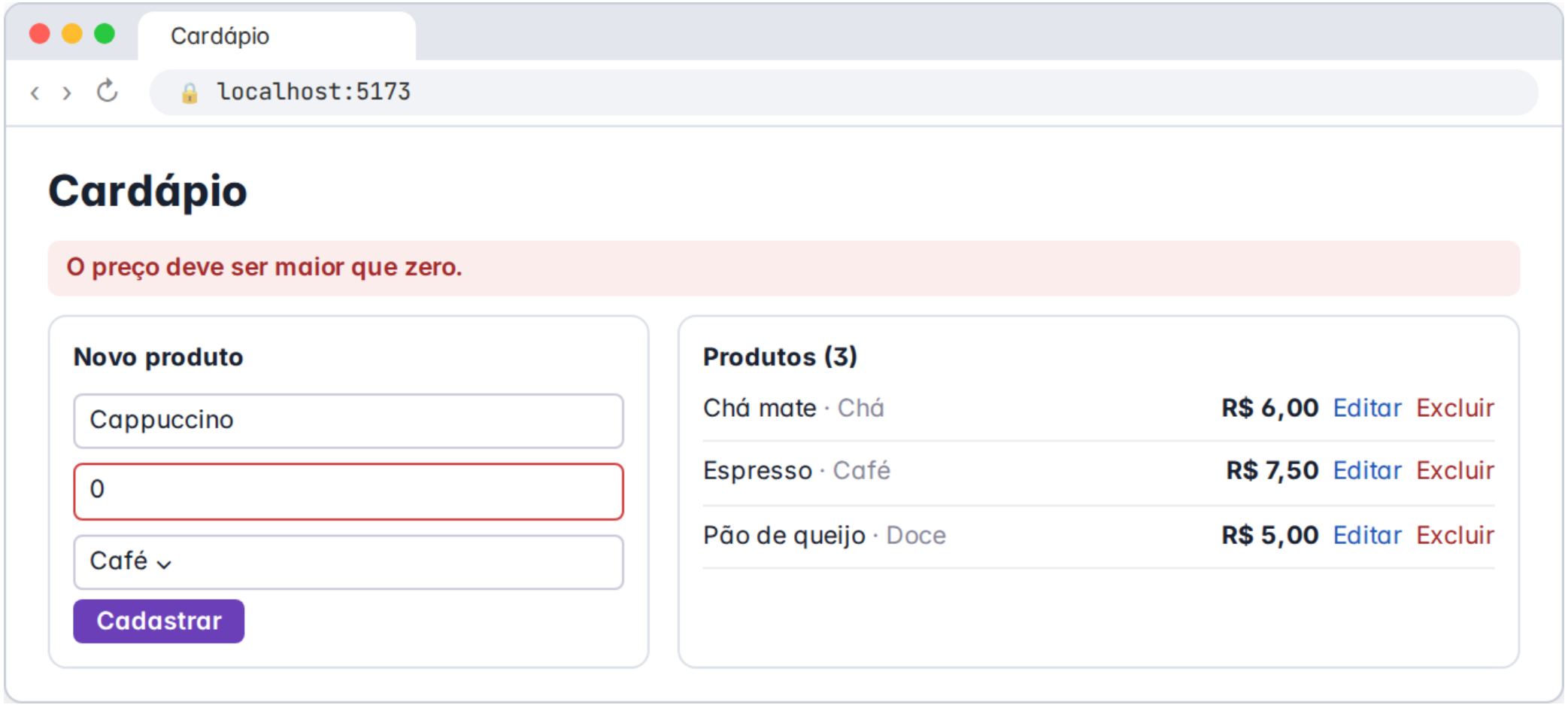
  return (
    <form onSubmit={enviar} className="card">
      <h2>{emEdicao ? "Editar produto" : "Novo produto"}</h2>
      <input name="nome" placeholder="Nome" value={form.nome} onChange={mudar} />
      <input name="preco" type="number" step="0.01" placeholder="Preço" value={form.preco} onChange={mudar} />
      <select name="categoria" value={form.categoria} onChange={mudar}>
        <option value="CAFE">Café</option><option value="CHA">Chá</option><option value="DOCE">Doce</option>
      </select>
      <label><input type="checkbox" name="disponivel" checked={form.disponivel} onChange={mudar} /> Disponível</label>
      <button type="submit">{emEdicao ? "Salvar alterações" : "Cadastrar"}</button>
      {emEdicao && <button type="button" onClick={onCancelar}>Cancelar</button>}
    </form>
  );
}
```

Passo 8: o App junta tudo



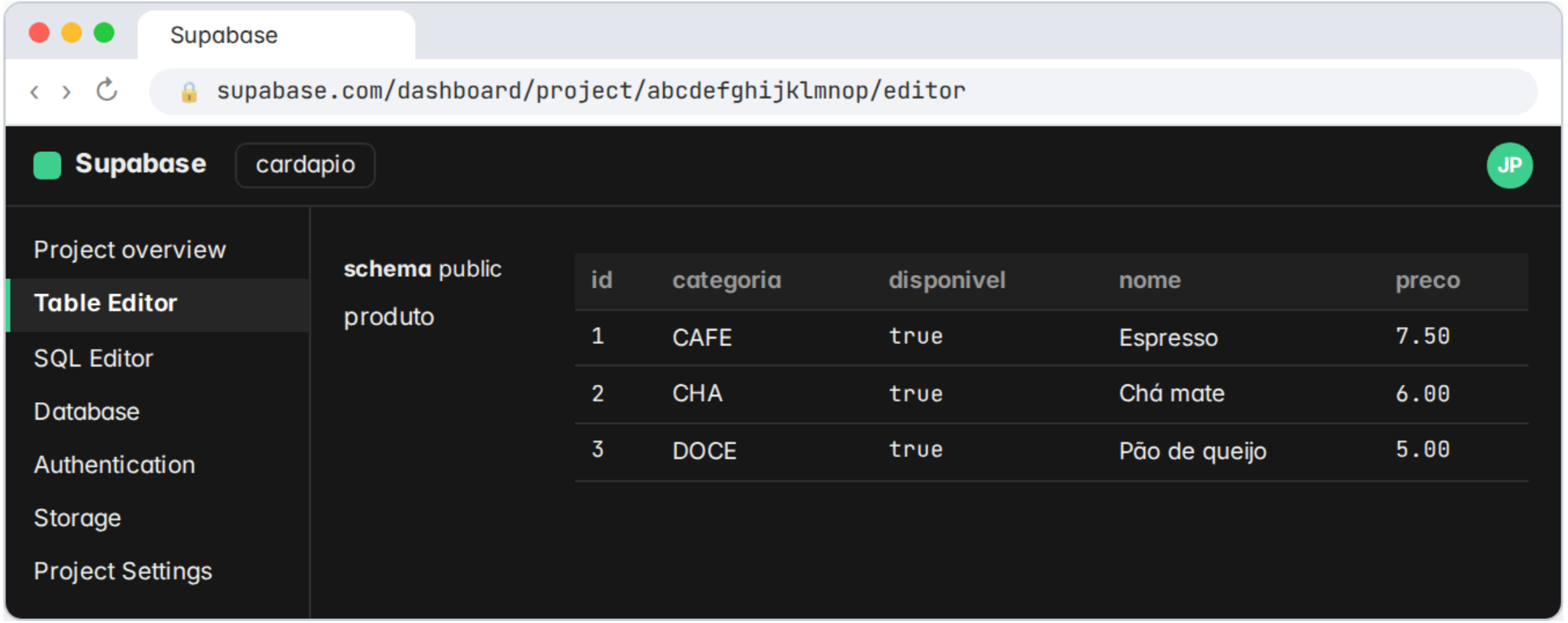
`App.jsx` (imports omitidos): o estado mora aqui; o formulário e a lista recebem dados e funções.

Passo 9: rodando e conferindo



O front mostrando a `mensagem` de validação que veio do `GlobalExceptionHandler` do Spring.

Passo 9: rodando e conferindo



Os mesmos produtos na tabela `produto`, criada pelo Hibernate no Supabase.

04

Aprofundando: deixando o CRUD profissional

Aprofundando: deixando o CRUD profissional

Busca + paginação no back

```java
@Query("select p from Produto p where lower(p.nome) like lower(concat('%', :q, '%'))")
Page<Produto> buscar(@Param("q") String q, Pageable pageable);

@GetMapping
public Page<Produto> listar(@RequestParam(defaultValue = "") String q,
                            @PageableDefault(size = 10, sort = "nome") Pageable p) {
    return repo.buscar(q, p);
}
```

Aprofundando: deixando o CRUD profissional

Paginação no front

```
const [pagina, setPagina] = useState(0);
const [dados, setDados] = useState({ content: [], totalPages: 0 });

useEffect(() => {
  requisicao(`/api/produtos?q=${encodeURIComponent(busca)}&page=${pagina}&size=10`).then(setDados);
}, [busca, pagina]);

<button disabled={pagina === 0} onClick={() => setPagina(pagina - 1)}>Anterior</button>
<span>Página {pagina + 1} de {dados.totalPages}</span>
<button disabled={pagina + 1 >= dados.totalPages} onClick={() => setPagina(pagina +
1)}>Próxima</button>
```

Atualização pessimista x otimista

Pessimista

- Espera o back responder e recarrega a lista
- Simples e sempre correta
- Usada no passo a passo da aula

vs

Otimista

- Atualiza a tela na hora e desfaz se der erro
- Sensação de app instantâneo
- Mais código para tratar a falha

Aprofundando: deixando o CRUD profissional

Exclusão otimista

```
const excluir = async (p) => {
  const antes = produtos;
  setProdutos(produtos.filter((x) => x.id !== p.id));    // some na hora
  try {
    await excluirProduto(p.id);
  } catch (e) {
    setProdutos(antes);                                   // volta se falhou
    setMensagem({ tipo: "erro", texto: e.message });
  }
};
```

Aprofundando: deixando o CRUD profissional

Dinheiro: BigDecimal no back, Intl no front

```
// formatação de moeda no front
const real = new Intl.NumberFormat("pt-BR", { style: "currency", currency: "BRL" });
<span>{real.format(produto.preco)}</span>        // R$ 7,50
```

Checklist do CRUD

Validação nas duas pontas

Front para a experiência; back para a segurança.

Estados da tela

Carregando, vazio, erro, sucesso.

Confirmação

Excluir pede confirmação; mostra o nome do item.

Feedback

Mensagem de sucesso some sozinha após alguns segundos.

Aprofundando: deixando o CRUD profissional

Na prática

O `AraCoffee` original carrega um cardápio inicial na primeira execução (`config/CardapioInicial.java`, um `CommandLineRunner`). É uma ótima técnica para demonstrações: o banco nunca começa vazio.

05

Quiz de revisão

Pergunta

Qual a ordem recomendada para construir o CRUD?

A Front e depois back

B Back testado sozinho e depois front

C Só o front

D Banco por último

Quiz · questão 1 de 5

Resposta

Qual a ordem recomendada para construir o CRUD?

A Front e depois back

✓ Back testado sozinho e depois front

C Só o front

D Banco por último

Por quê: Se o back não funciona com curl, no React também não.

Pergunta

Como um único formulário serve para criar e editar?

A Dois formulários

B Com o estado `emEdicao` definindo POST ou PUT

C Recarregando a página

D Com CSS

Resposta

Como um único formulário serve para criar e editar?

A Dois formulários

✓ Com o estado `emEdicao` definindo POST ou PUT

C Recarregando a página

D Com CSS

Por quê: useEffect preenche o formulário ao escolher um item.

Quiz · questão 3 de 5

Pergunta

Qual anotação ativa a validação do DTO no controller?

A @Validated no service

B @Valid no parâmetro @RequestBody

C @Entity

D @Schema

Resposta

Qual anotação ativa a validação do DTO no controller?

A @Validated no service

✓ @Valid no parâmetro @RequestBody

C @Entity

D @Schema

Por quê: Falhas viram MethodArgumentNotValidException.

Pergunta

Qual tipo usar para preço no back?

A double

B BigDecimal

C String

D float

Resposta

Qual tipo usar para preço no back?

A double

✓ BigDecimal

C String

D float

Por quê: Formate no front com Intl.NumberFormat.

Pergunta

O que é atualização otimista?

A Esperar o back

B Atualizar a tela antes da resposta e desfazer se falhar

C Não validar

D Usar cache

Quiz · questão 5 de 5

Resposta

O que é atualização otimista?

A Esperar o back

✓ Atualizar a tela antes da resposta e desfazer se falhar

C Não validar

D Usar cache

Por quê: Dá sensação de app instantâneo.

Fechamento

Resumo da aula

- ✓ Back primeiro: entidade → repository → DTO validado → service → controller → CORS → testar com curl.
- ✓ Front depois: apiConfig + httpClient → service → formulário/lista → App como dono do estado.
- ✓ Um formulário serve para criar e editar, controlado por `emEdicao`.
- ✓ Erros de validação viajam como JSON (`mensagem`) e o front só exibe.
- ✓ Confira sempre o resultado no banco (Table Editor).

Exercícios

1. Construa o cardápio completo e grave um vídeo curto mostrando o CRUD.

2. Adicione filtro por categoria no front usando `?categoria=` do back.

3. Adicione o campo `descricao` (opcional, até 200 caracteres) nas duas pontas.

4. Troque o banco para MongoDB Atlas alterando só `model`, `repository` e `application.properties`.

Para ir além

- Projeto `FullStack/AraCoffee` (versão completa com MongoDB, testes e deploy)

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 05?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web